DOCUNEXA

Backend Product Requirements Document & Sprint Execution Plan

Enterprise Document Intelligence Platform

Document control	Value
Version	1.0 — execution baseline derived from supplied project package
Prepared on	09 October 2026
Primary source materials	docs.zip: DocNexaReport.md, IMPLEMENTATION_STATUS.md, BACKEND_API_CONTRACT.md, architecture docs, schemas, traceability matrix, issues register; BRD v1.0
Planning focus	Backend APIs, persistence, async processing, AI pipeline, security, integrations, operations, release
Audience	Product owner, backend engineers, AI/ML engineers, QA, DevOps, frontend/mobile engineers
Status	Proposed execution plan; stack/policy conflicts listed for explicit sign-off

Planning note: the supplied project materials describe the frontend as complete in mock mode, while the backend is specified/designed and the production HTTP integration remains outstanding. This PRD therefore treats UI screens as existing work and prioritizes backend implementation, integration, verification, and release readiness.

1. Executive Summary

DocuNexa is an enterprise document-intelligence platform that accepts business documents, validates and stores originals, extracts text and structured fields, routes uncertain results to human reviewers, enforces configurable approvals, publishes approved records, supports search and version comparison, answers questions with evidence-backed citations, and manages retention, legal hold, exports, audit, and integrations.

The supplied implementation status reports Angular 19 screens completed and verified in mock mode, a PostgreSQL schema drafted, API contracts specified, and native iOS architecture documented. The audit also identifies mock services as the principal frontend integration gap, token-refresh behavior as a security gap, the document viewer as needing a production PDF renderer, and client-side report export as needing a server-authorized export flow.

- Primary objective: move from a convincing UI/prototype and documented architecture to a secure, persistent, observable, testable backend that supports real end-to-end workflows.
- Release must not be declared based only on successful frontend compilation. It requires authorization tests, persistence checks, queue retry/idempotency tests, AI evaluation evidence, UAT, backup/restore rehearsal, and security sign-off.
- AI outputs are advisory proposals. Human reviewers and authorized approvers retain control of material corrections and publication; AI must never approve, delete, sign, or post data autonomously.
- The initial supported document types are supplier contract, purchase invoice, and internal policy, with English test corpus as the proposed initial baseline unless the product owner approves otherwise.

1.1 Product outcomes

Outcome	Measure / proof
Reliable intake	Upload acknowledgement and durable intake record; scan/checksum/provenance; clear status and failure reason.
Trustworthy extraction	Field values carry source version, page/coordinates, model/config version, confidence, and review disposition.
Controlled publication	Required fields/exceptions resolved; approval route and separation-of-duties rules enforced server-side.
Permission-safe discovery	Search, snippets, downloads, Q&A, exports, and citations apply the same tenant/workspace/document/field authorization.
Auditable lifecycle	Version history, corrections, decisions, exports, integration attempts, legal holds, and deletion evidence are traceable.
Operational resilience	Idempotent jobs, bounded retries, dead-letter handling, health metrics, alerts, backup/restore, and rollback runbooks.

1.2 In scope

- Tenant and identity management, RBAC/MFA/SSO integration points, workspace and field-level access controls.
- Versioned document-type templates, rules, approval routes, retention policies, and schema validation.
- Upload API, file validation, malware quarantine, SHA-256 checksum, private object storage, duplicate candidates, versioning, and intake status.
- Durable asynchronous processing pipeline: rendering, OCR/layout, classification, extraction, deterministic validations, indexing, notifications, retries, and dead-letter workflow.
- Human review and correction history, exception/task management, approval workflow, publication, repository, search, comparison, grounded Q&A, reports/exports, webhooks, retention and purge.
- Frontend HttpClient integration and backend contract verification; backend support for native iOS needs such as approvals and notifications where included in the product release.

1.3 Out of scope for initial release

- Electronic signatures, payment initiation, autonomous negotiation/legal advice, and unapproved direct posting to finance/accounting systems.
- Unrestricted web crawling or ingestion from unapproved repositories.
- Production claims for handwriting, multilingual support, or scanned engineering drawings without separate labeled evaluation.
- Automatic legal conclusions or autonomous approval/deletion/posting by an AI service.

2. Source-Based Current State & Audit Interpretation

Area	What supplied documents say	Execution implication
Angular web/admin	IMPLEMENTATION_STATUS.md reports 14 screens complete and build verified in mock mode; DocNexaReport.md reports a successful Angular 19 build.	Preserve UI; replace Mock* services with typed HTTP services and validate real API behavior.
Backend	IMPLEMENTATION_STATUS.md calls Node.js API/workers specified/designed; backend API contract exists.	Treat backend as not production-complete until actual handlers, persistence, auth middleware, jobs, and tests are demonstrated.
PostgreSQL	DDL exists for tenant/users/documents/fields/audit/telemetry; status mentions PostgreSQL 16 and pgvector.	Review and normalize schema; add migrations, constraints, tenant isolation, transactions, indexes, rollback strategy.
MongoDB / Go docs	BRD and BACKEND_DEVELOPMENT_AND_GO_MICROSERVICES.md describe Go + MongoDB + Redis + S3, while the implementation status and other project reports describe Node.js + PostgreSQL/pgvector.	This is a material architecture conflict; resolve before sprint implementation and record an ADR.
Frontend security	Technical debt register says auth guard only checks token presence and lacks expiry/refresh handling.	Implement secure session lifecycle and never rely on route guards as backend authorization.
Document viewer	Technical debt register flags placeholder/layout-based PDF rendering rather than a production PDF.js viewer.	Integrate safe PDF rendering and page/coordinate mapping during frontend integration sprint.
Exports	Technical debt register says CSV is generated client-side.	Move exports to backend jobs/authorized streaming with audit events, masking, bounded file sizes and signed downloads.
iOS	Native Swift/SwiftUI app is documented as architecture/blueprint, not proven complete.	Treat iOS as a parallel track or later release gate depending on the committed product scope.

Evidence discipline: status labels in reports are treated as reported status, not independent proof. Every feature moves to “verified” only after code review, repeatable tests, and acceptance evidence are attached to the relevant requirement IDs.

3. Decisions Required Before Implementation

The supplied materials do not define one fully consistent implementation baseline. The following decisions are release-blocking architecture decisions; do not silently mix the alternatives.

Decision	Conflicting evidence	Recommended baseline for this plan	Required action
Backend language/framework	Node.js/TypeScript API in IMPLEMENTATION_STATUS.md and DocNexaReport.md vs Go/Gin/Fiber microservices in BRD and backend Go spec.	Node.js + TypeScript backend, consistent with the project’s current implementation-status report and stated backend integration target. Prefer one modular service first; split workers/services only when operationally justified.	Product/technical owner signs ADR-001; update all specs and CI templates.
Derived document store	BRD specifies MongoDB for OCR/layout, extraction payloads, clause diffs, Q&A traces; implementation status references PostgreSQL + pgvector and a relational DDL.	PostgreSQL is transactional source of truth. Use pgvector only for approved embedding/search needs. Retain MongoDB only if a documented scale/schema requirement justifies it.	Choose one canonical storage plan; document ownership, backup, deletion, and consistency boundaries.
Queue/cache	Go spec says Redis 7.2; functional requirements mention BullMQ for Node.js. BRD requires a durable queue but leaves vendor open.	If Node.js is selected, use a durable queue implementation supported by the deployed stack; BullMQ/Redis is a candidate, not an assumed fact.	Prove persistence, retries, deduplication, DLQ, visibility, and recovery in a spike.
Document types	API examples mention loan agreement/claim form/ID card; BRD initial baseline is supplier contract, purchase invoice, internal policy.	Use BRD’s three types as the MVP baseline.	Update OpenAPI enums, schema templates, fixtures and UI options.
SLOs	BRD proposes 99.5% uptime, P95 metadata/search <1s, 95% of 20-page docs within 5 minutes, RPO 24h/RTO 8h; NFR file suggests stricter values.	Use BRD values as provisional minimums until the product owner approves one NFR baseline. Do not advertise the stricter values without test evidence.	Approve load, data volume, concurrency, regions, languages, RPO/RTO and availability targets.
Retention/deletion policy	BRD says periods/jurisdictions remain open and legal/business owner must supply rules.	Policy engine configurable; no invented production retention duration.	Obtain retention schedule, hold authority, backup expiry policy and deletion evidence requirements.

Other baseline choices: PostgreSQL owns workflow state and the latest approved structured record; object storage owns binary bytes; all derived data and indexes are rebuildable and must be re-authorized against current primary state before display. Original files are immutable; replacement uploads create a new source version and restart review.

4. Personas, Roles & Authorization Requirements

Role	Primary capabilities	Mandatory boundaries
Platform operator	Provision tenants and monitor platform health.	No routine document-content access; privileged access is time-bound and audited.
Organization admin	Manage tenant users, roles, templates, approval routes, integrations and policies.	Own tenant only; cannot bypass workflow or tenant boundaries.
Uploader/contributor	Upload documents; view permitted submissions and statuses.	No approval; no global export; scoped to allowed workspace/items.
Reviewer	Correct fields, resolve exceptions, comment, assign/clarify, submit for approval.	Only assigned document types/workspaces; all edits retain before/after values and actor.
Approver	Approve, reject, or return with reason.	Assigned approval step only; configurable separation of duties.
Auditor/reader	Read approved content, inspect audit and export where authorized.	Read-only scope, masking, and field permissions apply.
Service identity / worker	Perform a narrowly scoped job such as OCR, indexing, notifications or connector delivery.	Cannot invoke human decision endpoints; least privilege; every job has tenant/document/correlation context.

Authorization must be enforced on the backend for every resource and action, not only in Angular routing. The same permission filter applies to document details, file URLs, page renditions, search results/snippets, exports, AI retrieval, citation click-through, and asynchronous jobs.

5. Target Architecture & Data Ownership

Layer	Responsibility	Rules
Angular 19 web/admin	Existing role-based UI, review workbench, approvals, compare, search, Q&A, reports and admin.	Use typed API contracts, centralized error handling, loading/empty/retry states, token/session handling, and no business-truth mock fallbacks in production.
Node.js/TypeScript API (decision pending ADR)	REST API, authentication integration, policy checks, domain services, workflow transitions, API versioning.	Modular boundaries; request validation; OpenAPI; correlation IDs; rate limits; consistent errors.
Async workers	OCR, classification, extraction, indexing, notifications, exports, retention/purge and webhook delivery.	Durable queue, idempotent handlers, bounded retries, DLQ, cancellation/timeout, observable job state.
PostgreSQL	Tenants, users/memberships, policies, templates, documents/source versions, accepted fields, tasks, approval decisions, audit events, outbox, deletion cases.	Transactional source of truth; migrations; foreign keys; tenant_id; indexes; optional RLS; safe transaction boundaries.
MongoDB (only if ADR approves)	Derived OCR/layout and large versioned analysis payloads, per BRD.	Schema validation and indexes; never primary approval truth; every read re-checks source version and current authorization.
Private S3-compatible object storage	Immutable originals, page renditions and export artifacts.	Private buckets; encryption; versioned immutable keys; checksum; signed short-lived URLs; no public object URLs.
Queue/cache	Durable asynchronous execution and delivery state.	Persistence, dedupe keys, retry/backoff, DLQ, visibility timeout, replay controls.
AI/OCR provider adapters	OCR/layout, classification, extraction, clause analysis, embeddings and Q&A.	Provider-neutral interface; explicit data policy; prompt-injection defense; evidence trace; timeouts and manual fallback.
Observability/operations	Logs, metrics, traces, health/readiness, alerts, deployment and recovery.	No secrets or document text in general logs; tenant/correlation dimensions; operational runbooks.

5.1 Core domain invariants

- Tenant isolation is mandatory on every tenant-owned record, derived artifact, job, cache key, storage key, search hit and export.
- Source file bytes are immutable; a replacement creates a new source version and invalidates stale extraction/index/Q&A results for current use.
- PostgreSQL owns workflow status and approved structured values; indexes and AI outputs are derived, versioned and rebuildable.

- No publication while mandatory fields or blocking exceptions remain unresolved, unless an explicit authorized override policy exists and is audited.
- A cited answer must be supported by the cited passage; if evidence is missing, inaccessible, stale or conflicting, the system abstains or flags uncertainty.
- Retention purge checks legal hold first and records completion/exception evidence across every store; hold suspends purge without restoring user access.
- AI service identities cannot approve, delete, sign, or post records.

6. API Contract & Module Requirements

API group	Core operations	Important controls
/api/v1/auth	login, refresh, logout, MFA enrollment/verification, password reset, SSO callback/session exchange	Short-lived access tokens, refresh rotation/revocation, rate limiting, secure cookies where applicable, MFA for privileged roles.
/api/v1/tenants and /users	tenant setup, invitations, memberships, roles, deactivation	Tenant-scoped admin policy, invite expiry, deactivated-user immediate denial.
/api/v1/document-types	create/update draft, publish version, schema/rules, review route, retention class	Immutable published versions; validate schema and route before publish.
/api/v1/documents	upload/session, list/search metadata, detail, versions, download/rendition, duplicate candidates	MIME/content validation, size/page limits, malware scan, checksum, tenant scope, short-lived signed URLs.
/api/v1/jobs	job status, retry, cancel where safe, operator inspection	Idempotency, authorized retry, attempt history, DLQ reason, correlation ID.
/api/v1/extractions	run status, field evidence, corrections, validation results	Source-version check; actor-bound edit history; confidence and provenance.
/api/v1/reviews and /tasks	assignment, comments, due dates, exceptions, clarification, submit review	State-machine validation, assigned role, optimistic concurrency, audit trail.
/api/v1/approvals	queue, approve, reject, return, decision history, publish	Assigned step only, mandatory reason for reject/return, SoD, no unresolved blocking exception.
/api/v1/search	metadata/full-text filters, saved searches, snippets	Permission-filtered results/snippets; no stale or deleted records.
/api/v1/compare	version-to-version text/field/clause changes	Both source versions authorized; cite both sides; uncertain alignment visible.
/api/v1/qa	question, scoped version selection, answer, citations, trace/feedback	Permission check at retrieval and citation click time; abstention; injection defense; no cross-tenant leakage.
/api/v1/reports and /exports	KPI, volume/aging/exception/accuracy/approval/retention reports, async export	Filter totals match visible rows; masking; audit; bounded exports; signed artifact download.
/api/v1/notifications and /webhooks	templates, delivery status/retry, webhook subscriptions	Recipient authorization; versioned templates; HMAC/signature; timestamp/replay guard; idempotent retries.
/api/v1/retention	policies, legal holds, deletion cases, purge status/evidence	Hold-first check; multi-store deletion; exceptions; backup expiry according to approved policy.

API standards required for all groups: OpenAPI contract, request/response schemas, pagination and sort allowlists, stable error envelope, validation errors, idempotency key support for mutating/retryable operations, correlation ID, UTC timestamps, authentication/authorization requirements, and contract tests.

7. Backend Sprint Plan

Recommended sequence: 13 delivery sprints plus a parallel iOS lane. Sprint durations are planning estimates, not commitments; the BRD states calendar estimates should follow dependency and volume baselining. If the 22 October 2026 target is fixed, treat it as a compressed release target with scope triage and explicit risk acceptance, not as proof that all capabilities can be production-hardened by that date.

Sprint	Focus	Priority	Primary exit
S0	Architecture & requirement baseline	P0 / blocking	Signed stack ADR, traceability, policies, test corpus and environment.
S1	Repository, CI/CD, runtime & observability	P0	Reproducible backend build, migrations, health checks, secrets and telemetry.
S2	Identity, tenant isolation & RBAC	P0	Negative cross-tenant/authz tests pass.

Sprint	Focus	Priority	Primary exit
S3	Document types, schema rules & workflow configuration	P0	Versioned template and policy publishing works.
S4	Secure intake, storage & versioning	P0	Upload → quarantine/validated → durable record and private object.
S5	Durable jobs, OCR & processing status	P0	Retryable asynchronous processing with DLQ and page evidence.
S6	Classification, extraction & deterministic validation	P0	Versioned candidate fields with evidence and rule exceptions.
S7	Review, correction, tasks & exception lifecycle	P0	Human corrections are persisted and auditable.
S8	Approval workflow & publication	P0	SoD and approval gates enforced by backend.
S9	Repository, versioning, search & comparison	P0	Permission-aware discovery and two-version comparison.
S10	Grounded Q&A / AI safety / evaluation	P0	Cited answers or abstention; injection and access tests pass.
S11	Notifications, reports, exports & webhooks	P1/P0 subset	Audited, bounded exports and reliable delivery.
S12	Retention, legal hold & secure purge	P0	Hold-aware multi-store deletion with evidence.
S13	Integration, security, performance, UAT & release	P0	All release gates met or documented exceptions approved.

Sprint 0 — Architecture & Requirements Baseline

Objective: Remove contradictions and freeze a buildable MVP baseline before parallel implementation.

Implementation tasks

Review BRD v1.0, audit report, implementation status, API contract, SQL/Mongo schemas, known debt and traceability matrix.

Approve ADR-001 backend language/framework; ADR-002 canonical data stores and vector strategy; ADR-003 queue; ADR-004 object storage; ADR-005 identity/session strategy.

Freeze MVP document types as supplier contract, purchase invoice and internal policy unless product owner explicitly changes scope.

Resolve role names, status enum/state transitions, endpoint naming, pagination, error format, version IDs and API auth scopes.

Approve file/page limits, English-language pilot corpus, malware scanning approach, AI provider/data policy, retention/hold rules and provisional NFRs.

Create requirement-to-API-to-schema-to-test traceability mapping for FR-001–FR-023, AI-001–AI-009, NFR-01–NFR-08 and UAT-01–UAT-12.

Deliverables: Signed architecture decision records; MVP scope and policy baseline; API/schema compatibility matrix; Test corpus and acceptance catalogue

Sprint exit / acceptance criteria:

- No unresolved blocker-level stack conflict.
- All Must requirements have owner, endpoint/module, data entity and test evidence planned.
- Open policy decisions are assigned to named roles and dates.

Sprint 1 — Backend Foundation, CI/CD & Observability

Objective: Create a repeatable service foundation that can be safely deployed and diagnosed.

Implementation tasks

Create service/module structure with configuration validation, environment profiles and graceful shutdown.

Add Dockerfile and local compose for approved dependencies; separate dev/test/staging secrets.

Create migration runner and baseline migrations; add deterministic seed fixtures for tenants, roles and sample documents.

Add liveness/readiness endpoints, structured logs, metrics, distributed correlation/request IDs and trace propagation.

Add centralized error handling, request validation, rate-limit middleware, CORS policy and API version routing.

Set CI checks for formatting, linting, unit tests, dependency/security scanning, migration validation and build artifact generation.

Add deployment rollback notes, configuration inventory and secrets/key rotation procedure.

Deliverables: Bootable API service; CI pipeline; Migration framework; Health/metrics endpoints; Developer setup guide

Sprint exit / acceptance criteria:

- Fresh environment can build and start from documented instructions.
- Migrations apply to an empty database and can be validated against a disposable test DB.
- Secrets are not committed; logs do not expose tokens, document contents or credentials.

Dependencies: Sprint 0 decisions and access to dev/staging infrastructure.

Sprint 2 — Identity, Sessions, Tenancy & RBAC

Objective: Enforce identity and authorization on every backend operation.

Implementation tasks

Implement tenant, user, membership, role and permission models with tenant-aware uniqueness and foreign keys.

Implement login/session issuance, short-lived access tokens, refresh rotation/revocation, logout and password reset where applicable.

Add MFA hooks and privileged-role enforcement; implement SSO adapter boundary if SAML/OIDC provider is selected.

Implement authorization middleware/policy service with tenant, workspace, document type, document, action and sensitive-field scope.

Add deactivation and invite flows; ensure revoked users lose API, file, snippet, export, Q&A and job access immediately.

Add brute-force controls, login audit events, secure cookie/header policy and CSRF protections where cookie sessions are used.

Test object-level authorization and cross-tenant IDOR paths across every route.

Deliverables: Auth endpoints; Policy enforcement module; Tenant/membership migrations; Security test suite

Sprint exit / acceptance criteria:

- Unauthorized requests receive consistent 401/403 responses.
- Cross-tenant read/write/download/search/export/Q&A tests all deny access.
- Expired/revoked sessions cannot be reused; refresh rotation detects reuse.
- Privileged roles require approved MFA policy.

Dependencies: Sprint 1 database and service foundation.

Sprint 3 — Document Types, Templates & Policy Configuration

Objective: Make extraction schemas, validations, approval routes and retention policies configurable and versioned.

Implementation tasks

Implement CRUD for document-type drafts and immutable published schema versions.

Support field definitions: key, label, type, cardinality, requiredness, accepted values, masking, export permission and reviewer role.

Implement versioned validation rules, deterministic rules and cross-field rule definitions.

Implement approval-route configuration with sequential/parallel steps, role constraints and separation-of-duties policy.

Implement retention-class and access-policy references without hardcoding unapproved legal retention periods.

Validate template publication, prevent invalid schema/route combinations and retain template version on every intake.

Add admin audit events and API contract tests.

Deliverables: Document-type APIs; Versioned template model; Rules/approval policy model; Admin contract tests

Sprint exit / acceptance criteria:

- A new published template is used by new intake only.
- Existing documents continue referencing their original template version.
- Invalid templates/routes cannot be published.
- Policy changes are auditable and tenant-scoped.

Dependencies: Sprint 2 authorization framework and product-approved field dictionaries.

Sprint 4 — Secure Intake, Object Storage & Source Versioning

Objective: Accept files securely and create durable immutable source-version records.

Implementation tasks

Implement multipart upload or upload-session flow with size/page limits and content sniffing independent of filename.

Integrate malware scanning and quarantine states; block unscanned or failed-scan content from processing/download.

Calculate SHA-256 checksum, store private object under immutable version key, and persist object metadata in PostgreSQL.

Implement web/API intake source provenance and email adapter contract; ensure every source passes identical validation.

Create document and source-version records transactionally with outbox event for processing enqueue.

Implement duplicate candidate detection by checksum and business identifiers; require reviewer resolution and never silently overwrite.

Implement secure download/rendition access using short-lived signed URLs or authorized streaming, with audit events.

Implement status endpoint and upload idempotency handling.

Deliverables: Upload API; Private storage adapter; Quarantine/scan integration; Source-version model; Duplicate candidate service

Sprint exit / acceptance criteria:

- Supported clean file returns durable ID/status; unsafe file is quarantined/rejected with reason.
- Original checksum is reproducible and original bytes cannot be overwritten.
- Duplicate candidates are visible and require a decision.
- No public object URLs or cross-tenant file access.
- Outbox record and document state remain consistent under failures.

Dependencies: Sprints 1–3; approved storage and malware scanning choices.

Sprint 5 — Durable Queue, OCR/Layout & Processing Orchestration

Objective: Process intake asynchronously without losing work or corrupting document state.

Implementation tasks

Implement durable queue integration and transactional outbox dispatcher; define stable job IDs and idempotency keys.

Implement job lifecycle: queued, running, succeeded, retryable failure, terminal failure/DLQ, cancelled where safe.

Implement retry/backoff, max-attempt policy, poison-message handling, worker heartbeat/lease and operator retry endpoint.

Build provider-neutral document renderer/OCR adapter that returns page text, reading order, dimensions, coordinates and failed-page markers.

Persist OCR/layout output in the approved derived-data store with tenant/document/source/analysis version keys.

Record model/provider/config version, processing time, errors, page-level confidence and correlation IDs.

Expose processing status, attempt history, failed pages and retry controls to authorized users.

Create fixtures for native PDF, scanned PDF, rotation, multi-column, tables, low DPI and malformed files.

Deliverables: Queue worker framework; OCR adapter; Processing/job APIs; DLQ/operator visibility; OCR test fixtures

Sprint exit / acceptance criteria:

- API remains available when workers fail.
- Duplicate queue delivery does not duplicate state transitions or artifacts.
- Failed pages are visible and manually reviewable.
- Retry is safe and records each attempt.
- Job metrics and errors are observable without leaking document text.

Dependencies: Sprint 4 outbox and approved queue/OCR provider decisions.

Sprint 6 — Classification, Extraction & Deterministic Rules

Objective: Produce versioned AI proposals with source evidence and deterministic validation.

Implementation tasks

Implement classification adapter with confidence/uncertainty threshold and manual reclassification path.

Implement schema-driven extraction for fields and invoice line items; persist proposed values separately from accepted values.

Attach source version, page, bounding box/span, confidence, model version and extraction run ID to every candidate.

Implement deterministic invoice arithmetic, date ordering, required fields, type/format and cross-field validations.

Detect conflicting values and duplicates; surface exceptions rather than auto-resolving conflicts.

Implement re-run logic when reviewer changes document type or source version; mark previous results stale.

Capture field-level corrections and reviewer dispositions for AI quality measurement without using customer documents for shared training by default.

Create labeled benchmark and report OCR, classification, extraction and provenance metrics separately.

Deliverables: Classification/extraction services; Validation engine; Evidence API; Exception model; Benchmark harness

Sprint exit / acceptance criteria:

- Every proposed field has retrievable evidence or explicit “evidence unavailable” state.
- Uncertain/missing mandatory fields require review.
- Invoice totals are recomputed deterministically; mismatches block approval unless authorized override.
- New source version invalidates prior analysis for current use.
- No unsupported accuracy claim without benchmark results.

Dependencies: Sprint 5 OCR/layout evidence and Sprint 3 published schemas.

Sprint 7 — Human Review, Corrections, Tasks & Exceptions

Objective: Persist reviewer decisions and provide a complete auditable exception workflow.

Implementation tasks

Implement review queue, assignment, reassignment, due date, comments and clarification requests.

Implement field accept/edit/reject operations with validation and mandatory override reason where policy requires.

Store before/after values, source evidence reference, actor, timestamp, policy/template version and correlation ID.

Implement exception types, severity, blocking/non-blocking classification, ownership and resolution status.

Implement optimistic concurrency/version checks so two reviewers cannot silently overwrite each other.

Expose document detail including OCR page geometry, extraction run, field confidence, history and page rendition URL.

Connect Angular HttpClient services to real endpoints; replace Mock* service in the target path and add contract/error-state tests.

Use PDF.js or approved viewer for source pages and coordinate overlays; test rotated pages and zoom mapping.

Deliverables: Review/task/exception APIs; Correction history; Frontend HTTP integration slice; Viewer evidence mapping

Sprint exit / acceptance criteria:

- Correction is durable and history retains original proposal plus corrected value.
- Only authorized assignee/role can edit; concurrent edits return conflict rather than overwrite.
- Mandatory unresolved exceptions block submission.
- Frontend displays loading/error/retry states and never fabricates production records.

Dependencies: Sprints 4–6; frontend service contracts and PDF renderer.

Sprint 8 — Approval Workflow, Separation of Duties & Publication

Objective: Enforce approval as a backend state machine, not a UI-only stepper.

Implementation tasks

Implement sequential and parallel approval steps with policy-version snapshot and assigned approver validation.

Implement approve/reject/return-for-changes actions; require reasons for rejection/return and audit all decisions.

Enforce uploader/reviewer versus approver separation of duties according to tenant policy.

Block approval when mandatory fields or blocking exceptions remain unresolved unless a documented authorized override is recorded.

Implement idempotent decision endpoints and optimistic state/version checks to prevent double approval.

Publish approved structured version transactionally and emit outbox event for indexing/notifications/connectors.

Preserve prior published versions; replacement upload starts a new review cycle without erasing history.

Add tests for role escalation, parallel approval, retry duplicate, wrong step, self-approval and stale version.

Deliverables: Approval state machine; Decision and publication APIs; SoD enforcement; Approval integration tests

Sprint exit / acceptance criteria:

- Only assigned approver can advance a valid step.
- SoD violations are rejected by API even if UI is bypassed.
- Published record references source version, template/policy version and approval events.
- Retries cannot duplicate publication or downstream side effects.

Dependencies: Sprint 7 review completion and Sprint 3 workflow configuration.

Sprint 9 — Repository, Search, Version Comparison & Index Freshness

Objective: Deliver searchable approved content and reliable version-to-version differences.

Implementation tasks

Implement document repository filters, pagination, sort allowlist and metadata queries with indexes.

Implement PostgreSQL full-text baseline; add pgvector or dedicated search only after approved architecture/load evidence.

Apply tenant/document/field permissions before returning results and snippets; do not trust index permissions alone.

Implement indexing via outbox and versioned index records; handle retry, rebuild and deletion invalidation.

Implement side-by-side source version comparison for text, structured fields and clauses; preserve both evidence locations.

Mark uncertain clause alignment and never invent missing text; keep legal interpretation with authorized human reviewers.

Implement saved searches if in approved scope and audit export/search actions according to policy.

Test stale index, revoked user, superseded version, deleted document, and tenant-filter edge cases.

Deliverables: Repository/search APIs; Indexing worker; Compare API; Index rebuild command

Sprint exit / acceptance criteria:

- Search filters/snippets match accessible records only.
- Revoked user cannot retrieve a result through stale index.
- Comparison identifies added/removed/changed/uncertain spans and cites both versions.
- Derived indexes can be rebuilt from authoritative stores.

Dependencies: Sprints 2, 4, 6 and 8; approved vector/index architecture.

Sprint 10 — Grounded Q&A, Retrieval Security & AI Evaluation

Objective: Provide evidence-backed answers from only authorized, current document versions.

Implementation tasks

Implement retrieval pipeline using only selected and currently permitted source versions.

Re-check access at retrieval time and again when a citation is opened; reject stale, revoked or deleted source references.

Implement citation schema: document title/ID, source version, page, snippet/span and stable authorized click-through reference.

Implement answer grounding checks, evidence-to-claim mapping and abstention when evidence is insufficient or conflicting.

Treat all uploaded text as untrusted; defend against prompt injection that asks the model to bypass controls or reveal other tenants.

Record question/answer trace, retrieval IDs, model/prompt/config version, citations, timing, feedback and abstention reason under retention policy.

Ensure provider outage leaves upload/manual review functional and never permits AI to call approval/deletion/posting endpoints.

Evaluate citation correctness, answer faithfulness, abstention, cross-tenant leakage, injection resistance and sensitive data exposure.

Deliverables: Q&A API; Retrieval authorization filter; Citation resolver; AI trace/evaluation suite

Sprint exit / acceptance criteria:

- Every material answer claim is supported by accessible evidence or is clearly marked uncertain.
- Insufficient evidence causes abstention, not a fabricated answer.
- Cross-tenant prompt-injection test passes.
- Citation click revalidates current authorization.
- AI identity cannot invoke human decision endpoints.

Dependencies: Sprints 2, 5, 6 and 9; approved model/data-processing policy.

Sprint 11 — Notifications, Reports, Exports & Webhooks

Objective: Make operational reporting and external delivery reliable, scoped and auditable.

Implementation tasks

- Implement versioned email/in-app notification templates for receipt, failure, assignment, return, approval and expiry.
- Validate recipients at dispatch time; record queued/sent/retried/failed status and provider response without leaking content.
- Implement dashboard KPIs, volume by source/type/status, aging, exception reasons, extraction quality, approval audit and retention reports.
- Ensure report totals reconcile with underlying permission-filtered records and date/time-zone rules.
- Implement asynchronous export jobs with authorized filter snapshot, PII/financial masking, size limits, expiry and signed download.
- Record export request, actor, scope, filters, artifact checksum and download event; remove client-side-only export path.
- Implement signed outbound webhooks with timestamp/replay guard, idempotency, bounded retry, delivery log and dead-letter recovery.
- Create sandbox connector contract and explicit mapping review for ERP/finance/CLM; external posting only after approval and mapping sign-off.

Deliverables: Notification worker; Reports APIs; Export worker and download flow; Webhook dispatcher; Connector sandbox

Sprint exit / acceptance criteria:

- Notification attempts and terminal failures are visible.
- Export filters match UI filters; unauthorized fields are masked or omitted.
- Large exports do not block API requests.
- Webhook retry does not duplicate business side effects.
- No external posting occurs before approved publication.

Dependencies: Sprints 2, 4, 8 and 9; approved delivery providers and connector mapping.

Sprint 12 — Retention, Legal Hold & Secure Purge

Objective: Implement policy-driven lifecycle deletion across primary, derived and binary stores.

Implementation tasks

- Implement versioned retention classes and scheduler using policy-approved periods; do not hardcode unapproved legal durations.
- Implement deletion cases with requester, legal basis/reference, approval, affected source versions, status and audit trail.
- Check active legal holds before purge; hold freezes deletion while preserving existing access decisions.
- Revoke access and invalidate search/Q&A caches, snippets, vectors, OCR/layout, extraction payloads, renditions, exports and controlled derivatives.
- Delete private object versions under approved policy and capture provider response/checksum/reference as completion evidence.
- Handle backups according to approved expiry/cryptographic-erasure policy and document exceptions for immutable backups.
- Implement retryable purge orchestration with per-store completion state, failure alerting and operator reconciliation.
- Test hold-vs-delete race, concurrent requests, partial store failure, resumed purge after hold release and audit retention.

Deliverables: Retention APIs and scheduler; Legal hold manager; Purge orchestrator; Deletion evidence report

Sprint exit / acceptance criteria:

- No purge executes while an active hold applies.
- Every required store reports completion or a visible exception.
- Deleted content no longer appears in search, Q&A, downloads or exports.
- Purge is idempotent and auditable.
- Backup behavior matches signed policy.

Dependencies: Sprints 1–4; approved retention, legal-hold and backup policy.

Sprint 13 — Integration, Security, Performance, UAT & Release

Objective: Prove end-to-end behavior and release only against agreed acceptance criteria.

Implementation tasks

Run UAT-01–UAT-12 from the BRD using synthetic/licensed fixtures and capture evidence per scenario.

Run threat-focused tests for tenant isolation, revoked sessions, stale URLs/indexes, prompt injection, sensitive data leakage and export authorization.

Run load tests for agreed concurrency, upload sizes, metadata/search latency, queue backlog and processing throughput.

Reconcile PostgreSQL workflow state with derived OCR/analysis, object storage and search index; run orphan detection.

Perform backup/restore drill across PostgreSQL, MongoDB if retained, object storage and index rebuild; measure RPO/RTO.

Test migration forward/rollback, key rotation, secret rotation, queue outage, AI provider outage and worker crash recovery.

Run accessibility checks on frontend flows and API error-state integration; complete mobile/API compatibility if iOS is in release scope.

Close critical/high security defects, review dependency/license findings, publish runbooks, support escalation and incident response.

Hold release review: Must requirements signed off or exceptions formally accepted; document AI limitations and benchmark results.

Deliverables: UAT evidence pack; Security/performance report; Recovery drill report; Deployment/rollback runbook; Release sign-off

Sprint exit / acceptance criteria:

- All Must FR/AI/NFR requirements pass or have signed exception.
- No critical/high security issues remain open.
- UAT-01–UAT-12 pass with traceable evidence.
- AI targets and document-type limitations are documented; human fallback works.
- Restore/rollback and integration retry are rehearsed.

Dependencies: All prior sprints, approved release targets and nominated UAT owner.

8. Cross-Cutting Definition of Done

- Code reviewed; lint, unit, integration and contract tests pass in CI.
- OpenAPI contract, migration, configuration, operational logs/metrics and runbook updated.
- Tenant and role authorization tests include positive and negative cases; object-level checks are not delegated to UI.
- Mutating and retryable operations have idempotency/concurrency behavior documented and tested.
- Audit events include actor/service identity, tenant, resource, action, timestamp, policy/source version and correlation ID where applicable.
- Errors are actionable but do not leak secrets, stack traces, raw document content or other tenant data.
- No new critical/high dependency or security issue is accepted without documented risk approval.
- Acceptance evidence is linked to FR/AI/NFR/UAT IDs and reviewed by QA/product owner.
- Feature flags/configuration are safe by default; production has no implicit mock data or demo credentials.

9. QA Strategy & Acceptance Matrix

Test layer	Coverage	Evidence required
Unit	Validation rules, state machines, policy decisions, parsers, signature helpers, idempotency utilities.	Automated CI results and branch coverage for critical logic.
Integration	PostgreSQL transactions/migrations, object storage, queue/outbox, OCR adapter, notifications, webhooks, retention.	Repeatable tests with isolated dependencies and failure injection.
Contract	OpenAPI schemas, pagination, auth scopes, error envelope, versioning and frontend service models.	Contract test report; generated spec diff reviewed.
Security	Tenant isolation, IDOR, revoked user, token refresh/replay, upload abuse, signed URL expiry, export masking, prompt injection.	Threat test matrix and no critical/high unresolved findings.
AI evaluation	OCR/layout by scan quality, classification confusion, field precision/recall, provenance, clause-change recall, citation correctness, faithfulness, abstention.	Labeled corpus, versioned metrics, limitations and thresholds approved before release.
Performance/resilience	Upload acknowledgement, API P95, queue depth, 20-page processing, worker crashes, provider outage, restore drill.	Load profile, percentiles, error rate, recovery measurements and capacity assumptions.
UAT	BRD UAT-01 through UAT-12.	Scenario, fixture, actor/role, expected result, actual result, evidence link and sign-off.

9.1 BRD UAT mapping

UAT	Scenario focus	Core backend proof
UAT-01	Configure invoice template, reviewer, approver and validation.	Versioned template and policy snapshot; historical documents unchanged.
UAT-02	Upload clean invoice and scanned contract.	Scan, OCR, classification, extraction evidence and status.
UAT-03	Invoice mismatch and duplicate number.	Both exceptions visible; no silent publication/deletion.
UAT-04	Reviewer correction then approver publishes.	Before/after history and auditable publication.
UAT-05	Upload revised contract and compare.	Both source versions retained; changes cite both pages.
UAT-06	Ask renewal/payment question.	Authorized page/version citation or abstention.
UAT-07	Document contains malicious instructions/cross-tenant request.	Injection ignored; no cross-tenant evidence or citation.
UAT-08	Revoked user reuses old result, URL or export.	Execution-time denial; stale artifact cannot leak content.
UAT-09	OCR fails on several pages.	Failed pages marked; manual entry and safe retry.
UAT-10	Queue/email/ERP adapter fails then recovers.	Visible retries; no duplicate approval/posting.
UAT-11	Legal hold and deletion request target same document.	Purge pauses; approved release resumes with evidence.
UAT-12	Load and restore drill.	Agreed latency/recovery targets met; derived indexes rebuild.

10. Provisional Non-Functional Requirements & Release Gates

The BRD marks its numerical targets as provisional. These targets become binding only after product/operations approval. The separate NFR document contains stricter targets; do not combine them silently.

Metric	BRD baseline (provisional)	Verification
Availability	99.5% monthly user-facing availability excluding agreed maintenance.	Synthetic monitoring and monthly availability report.
Metadata/search latency	P95 under 1 second at agreed load.	Load test with agreed data volume, filters and concurrency.
Upload acknowledgement	Under 5 seconds for supported file size; processing asynchronous.	Upload test with status and queue monitoring.

Metric	BRD baseline (provisional)	Verification
Processing	95% of 20-page test documents within 5 minutes under agreed queue load.	Labeled processing benchmark by type and scan quality.
Recovery	RPO 24 hours and RTO 8 hours proposed.	Restore drill across primary/derived stores, files and index rebuild.
Security	No critical/high security defects open at release.	Security review and manual tests.
Traceability	Every job/state change traceable by tenant, document, actor and correlation ID.	Audit/log/trace inspection.
Operations	Migration, rollback, backup and key rotation rehearsed in staging.	Recorded release rehearsal.

The project NFR file also states more aggressive targets (for example, 99.9% uptime, shorter RPO/RTO, and sub-second API goals). Product and engineering must choose a single measurable baseline before performance work is considered complete.

11. Risks, Dependencies & Mitigations

Risk / dependency	Impact	Mitigation / owner action
Backend/storage stack contradiction	Rework, duplicate persistence, inconsistent contracts.	Resolve ADRs in Sprint 0; freeze stack before parallel coding.
Mock-first frontend mistaken for production readiness	Screens appear complete while no durable backend behavior exists.	Track UI completion separately from end-to-end acceptance; remove production mock fallback.
AI quality lacks agreed benchmark thresholds	Unsupported accuracy claims and poor reviewer trust.	Build labeled corpus and approve per-document-type thresholds before release.
Tenant isolation in derived data/indexes	Cross-tenant content leakage via snippets, citations or stale jobs.	Reauthorize on every read/click; tenant-scope job/cache/index keys; negative security tests.
Policy decisions unresolved	Retention, language, file limits and regions cannot be safely finalized.	Name product/legal/security owners and make release gates explicit.
Tight calendar to 22 Oct 2026	Full backend, AI, integration and hardening scope may exceed available capacity.	Prioritize critical-path MVP, parallelize independent work, demonstrate daily, and document any deferred Must requirement.
Provider/connector availability	OCR, email, SSO, object storage and ERP paths blocked.	Use provider adapters and sandbox contracts; never represent mock provider behavior as production integration.
Multi-store deletion/backup complexity	Residual data or unverifiable purge.	Define store-by-store deletion evidence and backup expiry policy; test partial failure/recovery.

12. Suggested Team Ownership & Parallel Tracks

Workstream	Accountability	Parallelism / handoff
Product/BA	MVP scope, policy decisions, field dictionaries, acceptance and change control.	Must resolve Sprint 0 choices and provide reviewers/approvers/test documents.
Backend/API	Domain modules, auth, endpoints, persistence, state machines, OpenAPI.	Owns API compatibility and migration strategy.
AI/document processing	OCR, classification, extraction, evaluation and grounded Q&A adapters.	Starts OCR spike after storage/queue interfaces are agreed; must provide evidence schemas.
QA/security	Requirement traceability, automation, adversarial tests, UAT evidence.	Test design starts Sprint 0; each sprint adds tests, not only final QA.
DevOps/SRE	CI/CD, secrets, containers, telemetry, backup/restore, deployment/rollback.	Foundation from Sprint 1; staging ready before integration/UAT.
Frontend	Replace mocks with HTTP services; auth lifecycle; PDF rendering; server exports.	Can integrate incrementally after each API contract is frozen.
iOS Swift	Native app API client, Keychain/session, approvals, push notifications.	Separate release lane; API contract stable first; do not block core web/backend MVP unless iOS is contractually required.

13. Delivery Timeline & Scope Strategy

The project report references 22 October 2026 as a backend/database engineering target. From the 9 October planning date, that is a short execution window for the full scope in this PRD. A realistic plan should use sprint goals as quality gates rather than promise that all 13 sprints fit into two weeks.

- If 22 October is a hard demo date: deliver a narrowly defined vertical slice—auth/tenant scope, secure upload, durable processing status, OCR/extraction proposal, reviewer correction, approval, repository and audit—using approved providers and test fixtures. Clearly label advanced Q&A, enterprise SSO, broad connectors, full purge across backups and scale targets if not verified.
- If production release is the goal: keep all sprints and do not waive security, authorization, AI evidence, retention policy or recovery gates to meet a calendar date.
- Use a daily integration build, sprint demo, and requirement evidence review. Every deferred requirement needs owner, rationale, risk, target sprint and product-owner acceptance.

14. Change Control & Traceability

- Every change to API, schema, policy, AI model/provider, retention behavior or workflow must include impact analysis for data, security, frontend, iOS, integrations, tests and migration/rollback.
- Maintain a traceability matrix mapping each BRD FR, AI requirement, NFR and UAT scenario to API/module, schema entity, test case and evidence artifact.
- Version OpenAPI and published document-type schemas. Breaking API changes require compatibility review and migration plan.
- Never edit historical approval decisions or source files in place; record corrections and supersession as new versioned events.
- A sprint is complete only when acceptance criteria and test evidence are reviewed, not when code is merely committed.

15. Final Release Checklist

- ■ Stack and data ownership ADRs approved; no unresolved architecture contradiction.
- ■ All Must requirements mapped and either passed or formally exception-approved.
- ■ All critical/high security issues closed; tenant isolation and revoked-access tests pass.
- ■ Uploads scanned, checksummed, private, versioned and retry-safe.
- ■ Processing queue is durable, idempotent, observable and has DLQ/recovery procedure.
- ■ AI extraction and Q&A have labeled evaluation results, citation verification, abstention and human fallback.
- ■ Review/approval state machine enforces role assignment, separation of duties and blocking exceptions server-side.
- ■ Search, citations, exports and file downloads re-check current permissions and source freshness.
- ■ Retention/legal hold/purge behavior follows approved policy and reports completion or exceptions across stores.
- ■ UAT-01–UAT-12 executed; integration retries do not duplicate publication or external posting.
- ■ Load, backup/restore, migration rollback, key rotation and deployment rollback rehearsed.
- ■ Runbooks, alerts, support ownership, release notes and product/security sign-off complete.

Appendix A — Source Materials Used

- Uploaded Document_Intelligence_Platform_BRD (1)(1).docx — BRD v1.0, 29 September 2026, 11 pages.
- docs.zip/docs/DocNexaReport.md — project and daily work master report dated 7 October 2026.
- docs.zip/docs/IMPLEMENTATION_STATUS.md — frontend mock-mode status, Node.js backend design status, PostgreSQL DDL status, iOS blueprint status.
- docs.zip/docs/BACKEND_API_CONTRACT.md — illustrative REST and WebSocket API contract.
- docs.zip/docs/BACKEND_DEVELOPMENT_AND_GO_MICROSERVICES.md — Go/MongoDB/Redis/S3 alternative architecture specification.
- docs.zip/docs/DATABASE_SCHEMA_POSTGRES.sql and DATABASE_SCHEMA_MONGODB.json — database schema drafts.
- docs.zip/docs/KNOWN_ISSUES_AND_TECHNICAL_DEBT.md — mock service, auth refresh, PDF viewer and client-side export findings.
- docs.zip/docs/BRD_TRACEABILITY_MATRIX.md, DOCUMENT_INTELLIGENCE_PLATFORM_BRD_ANALYSIS.md, ARCHITECTURE.md, ARCHITECTURE_DECISIONS.md, functional and non-functional requirements.

Source limitation: this plan is based on the supplied documentation package and BRD. It does not claim to have independently inspected the complete live repository or executed the backend. Actual code and test evidence should be checked during Sprint 0/1.